

Problems in RL

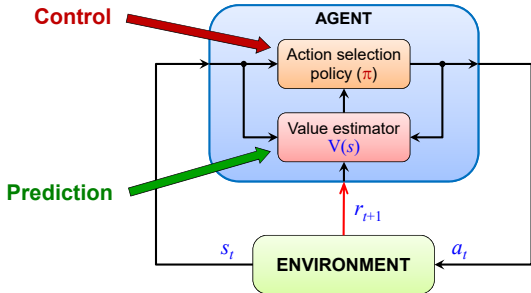
Prediction: (**evaluation** or **estimation**) it is the process of predicting the value function $V(s)$ that estimates the maximum return achievable in each state.

Control: it is the process of improving the policy (π), with the objective of reaching an optimal policy (π^*) that generates actions that maximize the return.

Planning: it is the process of evaluating actions in the future without taking them, but “simulating” them using a **model of the world** to reduce the interactions with the **real world**.

We first consider **model-free methods**, addressing how to predict state values.

Prediction vs. Control



Prediction methods

How to learn values?

How can the agent learn to predict $V(s_t) \approx R_t = r_{t+1} + R_{t+1}$ if we do not know the future?

Since actions are **stochastic**, different actions can be selected from the same state at different times, giving different rewards. The more we go into the future, the more R_t may diverge. Hence, **future rewards** are **discounted** by a factor $\gamma \in [0,1]$:

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \dots + \gamma^{T-1} r_T = \sum_{k=0}^{T-t-1} \gamma^k r_{t+1+k}$$

The **total discounted future reward** at step t is given by:

$$R_t = r_{t+1} + \gamma (r_{t+2} + \gamma r_{t+3} + \dots) = r_{t+1} + \gamma R_{t+1}$$

Total discounted future reward

The **total discounted future reward** at step t is given by:

$$R_t = r_{t+1} + \gamma (r_{t+2} + \gamma r_{t+3} + \dots) = r_{t+1} + \gamma R_{t+1}$$

Note that

- If $\gamma = 0$, then the strategy only relies on immediate rewards.
- If the environment is **deterministic** and the same actions always result in same rewards, then we can set $\gamma = 1$.
- In general, a value $0 < \gamma < 1$ (e.g., $\gamma = 0.9$) will balance between immediate and future rewards.

Goal of RL

GOAL of RL: discover **how to associate states to actions** to maximize the reward received in the long run, i.e., the **total discounted future reward** R_t

Total discounted future reward at step t :

Return or *cumulative discounted reward*

$$R_t = r_{t+1} + \gamma R_{t+1} \quad (\gamma \approx 0.9)$$

This can be achieved if we are able to associate each state s with a value $V(s)$ that predicts the *expected cumulative reward* achieved by following the policy π from state s :

$$V_{\pi}(s) = E \{R_t \mid s_0 = s, \pi\}$$

Value estimation

$$V(s_t) = \hat{R}_t = r_{t+1} + \gamma \hat{R}_{t+1}$$

Note that since r_{t+1} is known, the only term to predict is R_{t+1} .

But how can we predict R_{t+1} if we don't know the future?

There are several methods. The simplest one is to take an action, get the immediate reward r_{t+1} and go to the next state s_{t+1} :



If s_{t+1} is a **terminal state**, then there is no future return to be estimated (i.e., $R_{t+1} = 0$), hence the value of state s_t can be updated with $V_{target}(s_t) = R_t = r_{t+1}$.

Value estimation

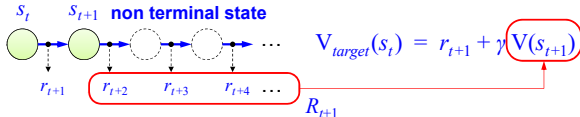
If s_{t+1} is **not a terminal state**, then R_{t+1} can be approximated by the value $V(s_{t+1})$ estimated so far:

$$\hat{R}_{t+1} \cong V(s_{t+1}) = V(T(s_t, a_t))$$

Hence, the value of state s_t can be updated with:

$$V_{\text{target}}(s_t) = r_{t+1} + \gamma V(s_{t+1})$$

Bellman equation



The process of computing the new estimate ($V_{\text{target}}(s_t)$) based on other estimates ($V(s_{t+1})$) is referred to as **bootstrapping**.

Value update

Since $V_{target}(s_t)$ is an estimate (i.e., could be wrong), the new value $V_{new}(s_t)$ for state s_t is computed as a weighted average between the new estimate $V_{target}(s_t)$ and the old value $V_{old}(s_t)$:

$$V_{new} = \alpha V_{target} + (1 - \alpha) V_{old}$$

where α is the weighting factor, or **learning rate**.

Note that the new value can also be expressed as

$$V_{new} = V_{old} + \alpha (V_{target} - V_{old})$$

The difference $\delta = V_{target} - V_{old}$ is referred to as **temporal difference error**, or **TD error** and this method is referred to as **TD learning**.

Incremental average

Note that, if $\alpha = 1/k$, the previous weighted average is a simple incremental average. In fact, given a sequence of values $x_1, x_2, \dots, x_k$ the average A_k is:

$$A_k = \frac{x_1 + x_2 + \dots + x_k}{k}$$

$$\begin{aligned} kA_k &= x_1 + x_2 + \dots + x_k = (x_1 + \dots + x_{k-1}) + x_k \\ &= (k-1)A_{k-1} + x_k = kA_{k-1} + (x_k - A_{k-1}) \end{aligned}$$

$$A_k = A_{k-1} + \frac{1}{k}(x_k - A_{k-1})$$

Incremental
average

Hence: $A_k = A_{k-1} + \alpha(x_k - A_{k-1})$ where $\alpha = \frac{1}{k}$

Exponential average

The simple incremental average

$$A_k = A_{k-1} + \alpha (x_k - A_{k-1}) = \frac{x_1 + x_2 + \dots + x_k}{k}$$

with $\alpha = 1/k$, assumes that all values have an equal weight.

However, in a changing environment, more recent values are more useful for estimating the current situation.

To give more weight to recent examples we can set α as a constant ($0 < \alpha \leq 1$) that does not depend on k :

$$A_k = A_{k-1} + \alpha (x_k - A_{k-1})$$

This is called an *exponential recency-weighted average*.

Exponential average

In fact, note that:

$$\begin{aligned}
 A_k &= A_{k-1} + \alpha (x_k - A_{k-1}) \\
 &= \alpha x_k + (1-\alpha)A_{k-1} \quad [A_{k-1} = \alpha x_{k-1} + (1-\alpha)A_{k-2}] \\
 &= \alpha x_k + (1-\alpha)\alpha x_{k-1} + (1-\alpha)^2 A_{k-2} \quad [A_{k-2} = \alpha x_{k-2} + (1-\alpha)A_{k-3}] \\
 &= \alpha x_k + (1-\alpha)\alpha x_{k-1} + (1-\alpha)^2 \alpha x_{k-2} + \dots + (1-\alpha)^k \alpha x_0 \quad [A_0 = \alpha x_0] \\
 &= \sum_{i=0}^k (1-\alpha)^{k-i} \alpha x_i
 \end{aligned}$$

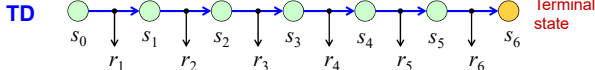
Since $(1 - \alpha) < 1$, the weight given to a sample x_i decreases exponentially with its age. For this reason, this is called an *exponential recency-weighted average*.

TD learning

To solve the prediction problem, the agent needs to estimate the **total discounted future reward** $\hat{R}_t$ in each state s_t :

$$\hat{R}_t = r_{t+1} + \gamma \hat{R}_{t+1}$$

To do so, it has to explore the world, starting from a **random state** s_0 and taking random actions until reaching a terminal state s_T .



Each time the agent visits a state s , it computes the TD error

$$\delta = V_{target}(s) - V_{old}(s)$$

and updates its value as $V_{new}(s) = V_{old}(s) + \alpha\delta$.

Note on initial values

The target value of a state s is computed as follows:

$$\hat{R}_t = V_{\text{target}}(s_t) = \begin{cases} r_{t+1} & \text{if } s_{t+1} \text{ is a terminal state} \\ r_{t+1} + \gamma V(s_{t+1}) & \text{otherwise} \end{cases}$$

Any arbitrary value can be used to initialize the $V(s)$ vector, but, if we initialize it with all **zeros**, we can use the same formula to compute the target value in any state:

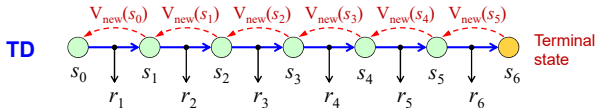
$$V_{\text{target}}(s_t) = r_{t+1} + \gamma V(s_{t+1})$$

In fact, since the value of a terminal state is never updated, $V(s_{t+1})$ will always be zero, if s_{t+1} is a terminal state, making the previous formula equivalent to the one in the yellow box.

Visualizing TD learning

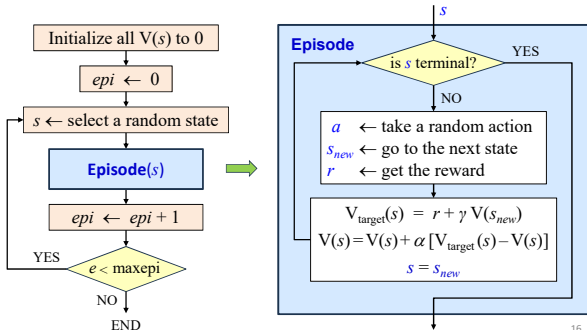
TD learning consists in exploring the world to learn the value of each state. At each a step, the agent

- takes a random action from the current state s
- go to the next state s_{new}
- receives the reward r
- updates the value of state s
- repeats the process until reaching a terminal state



TD learning algorithm

The **TD learning** algorithm can be implemented as follows:



TD pseudo code

```

TD_learning(max_episodes) {
  for (each s) V(s) = 0
  for (e=0 to max_episodes) {
    s = <select a random state>
    while (is_not_terminal(s)) {
      a = take_action(s)
      snew = next_state(s, a)
      r = get_reward(s, a, snew)
      Vtarget = r +  $\gamma$  * V[snew]
      V[s] = V[s] + alpha * (Vtarget - V[s])
      s = snew
    }
  }
}
  
```

Episode

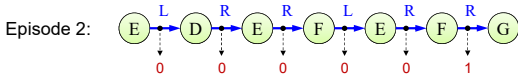
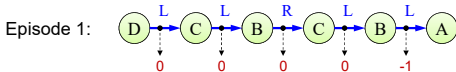
NOTE `take_action` can use a **random policy** to explore the world. To use the **ϵ -greedy policy**, the agent must have a model of the world in order to find the best action.

Example

Consider an environment with 7 rooms, where an agent can move making only two actions: **LEFT** and **RIGHT**. The reward is -1 going in room A, $+1$ in room G, and 0 in every other room.

Room :	A	B	C	D	E	F	G
Reward:	-1	0	0	0	0	0	1

An episode ends when reaching room A or room G.



Example

If $V(s)$ are initialized with 0 and the agent makes **random actions**, starting from a random state, after infinite episodes we get:

Room:	A	B	C	D	E	F	G
Reward:	-1	0	0	0	0	0	1
$V_{\infty}(s)$:	-1	-2/3	-1/3	0	1/3	2/3	1

We can run an experiment to see how the state values predicted by TD learning converge to $V_{\infty}(s)$ over a number of episodes.

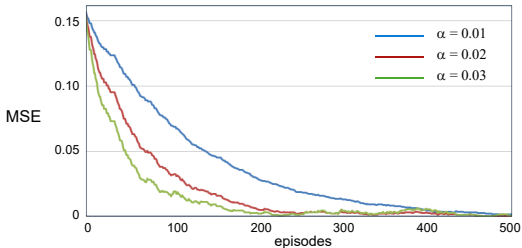
We can measure the **mean square error** (MSE) with respect to $V_{\infty}(s)$: at the end of each episode:

$$\text{MSE} = \frac{1}{5} \sum_{s=1}^5 [V(s) - V_{\infty}(s)]^2$$

Example

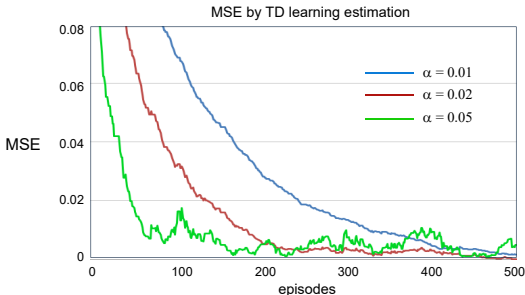
Room:	A	B	C	D	E	F	G
Reward:	-1	0	0	0	0	0	1
$V_{\infty}(s)$:	-1	-2/3	-1/3	0	1/3	2/3	1

The MSE varies depending on the learning rate α :



Example

Higher values of the learning rate α reduce the error more quickly, but introduce higher noise in $V(s)$ estimates:

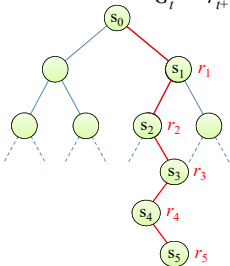


Monte Carlo method

Monte Carlo method

Monte Carlo method exploits random actions to reach the end of the episode from an initial state s_t and get the real total reward, here denoted as G_t :

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots + \gamma^{T-t-1} r_T = \sum_{k=0}^{T-t-1} \gamma^k r_{t+1+k}$$



end of episode

At the end of the episode, the values of the visited states are updated as

$$V(s_t) = V(s_t) + \alpha [G_t - V(s_t)]$$

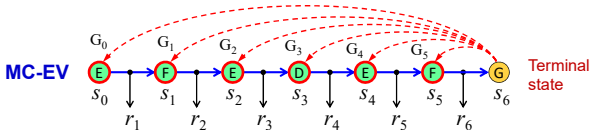
- Monte Carlo has **less bias** than TD learning, but the value estimation has a **higher noise**.
- Since here the future is not uncertain, we can use $\gamma = 1$.

Monte Carlo - Every Visit

There are two variants of the Monte Carlo method:

Monte Carlo – Every Visit

It updates all the states visited during each episode:



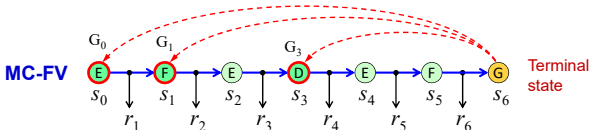
To implement this, you need to store all the states and rewards for the entire episode. If a state is visited more than once, its value is updated at **every visit**.

Monte Carlo - First Visit

There are two variants of the Monte Carlo method:

Monte Carlo – First Visit

It updates only the states visited for the first time in each episode:



To implement this, you need to store all the states and rewards for the entire episode. If a state is visited more than once, its value is updated only on the **first visit**.

Monte Carlo method

In both cases, the $V(s)$ is updated by $V(s) = V(s) + \alpha [G - V(s)]$ where G is the return from s to the terminal state s_T and α can be:

$$\alpha = \begin{cases} 1/N(s) & \text{to apply } \textit{incremental average}, \text{ where } N(s) \text{ is the} \\ & \text{number of visits of state } s \text{ since the first episode.} \\ \textit{constant} & \text{to apply the } \textit{exponential recency-weighted average}. \end{cases}$$

While in **MC-EV** this update is performed every time state s is visited in each episode, in **MC-FV** this update is performed only when state s is visited for the first time in each episode.

Hence, to implement **MC-FV**, we have to maintain another counter, $\text{visits}(s)$, that counts the number of times s is visited in an episode:

$$\begin{cases} N(s) & \text{is reset once before the first episode;} \\ \text{visits}(s) & \text{is reset before each episode.} \end{cases}$$

MC-EV pseudo code

```

Monte_Carlo_Every_Visit(max_episodes) {

    <reset  $V(s)$  and  $N(s)$  for every state>

    for (e=0 to max_episodes) {
         $s_0$  = <select a random state>;
        <generate episode from  $s_0$ : storing each  $(s, r)$ >
         $G = 0$ ;
        for ( $s=s_T$  to  $s_0$ ) {           // proceed backwards
             $G = \gamma G + r$ ;
             $N(s) = N(s) + 1$ ;
             $V(s) = V(s) + [G - V(s)]/N(s)$ ;
        }
    }
}
  
```

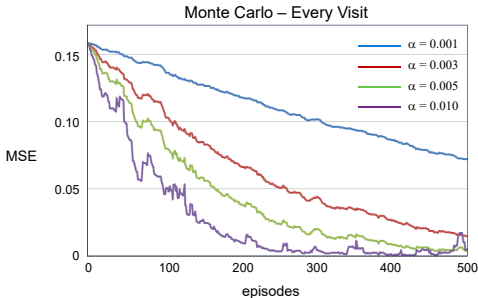
MC-FV pseudo code

```

Monte_Carlo_First_Visit(max_episodes) {
  <reset  $V(s)$  and  $N(s)$  for every state>
  for (e=0 to max_episodes) {
    <reset counter  $visits(s)$  for every state>
     $s_0$  = <select a random state>;
    <generate episode from  $s_0$ : storing each  $(s, r)$  and
    incrementing  $visits(s)$  each time  $s$  is visited>
     $G = 0$ ;
    for ( $s=s_T$  to  $s_0$ ) {      // proceed backwards
       $G = \gamma G + r$ ;
      if ( $visits(s) == 1$ ) {
         $N(s) = N(s) + 1$ ;
         $V(s) = V(s) + [G - V(s)]/N(s)$ ;
      }
       $visits(s) = visits(s) - 1$ ;
    }
  }
}
  
```

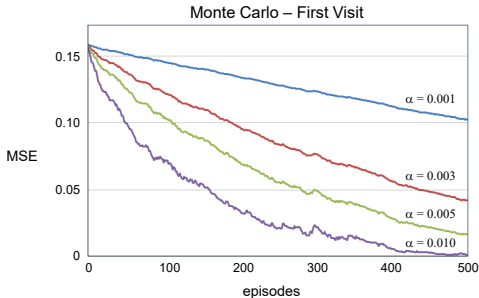
Example

Applying **Monte Carlo** methods to the previous example, we have to use a lower learning rate to get value estimates with less noise:



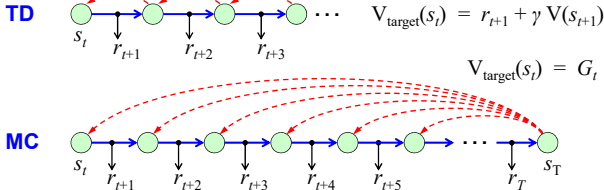
Example

Notice that **MC-FV** produces less noise with respect to **MC-EV**, since each visited state is updated only once per episode:



TD vs. Monte Carlo

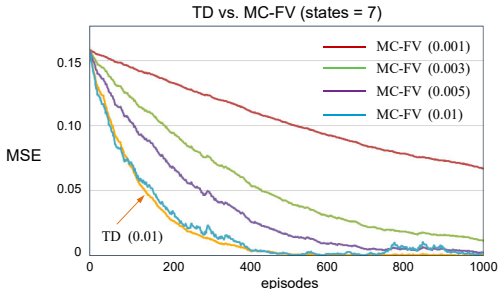
TD updates values at each step based on previous estimates, while MC updates values after an entire episode based on real rewards.



Note that in **TD learning** the new estimate ($V_{\text{target}}(s_t)$) is based on an other estimate ($V(s_{t+1})$). This is referred to as **bootstrapping**. Instead, in **MC**, the new estimate is based on a real return (G_t).

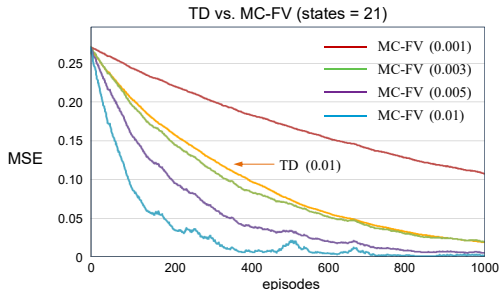
TD vs. Monte Carlo

For the simple problem we have considered (states = 7), we can see that the **TD** method works better than **MC-FV**:



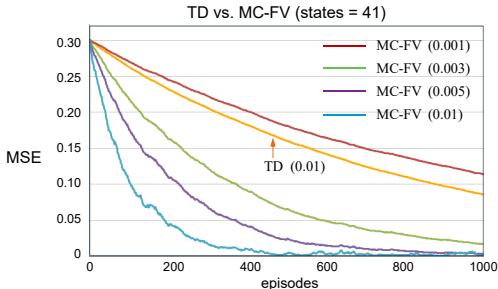
TD vs. Monte Carlo

If we increase the number of states (**states = 21**), the **TD** method starts losing performance:



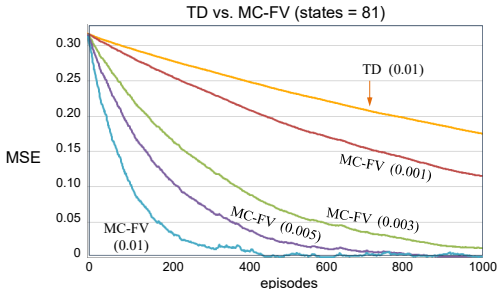
TD vs. Monte Carlo

If we further increase the number of states (**states = 41**), the **TD** performance continues to decrease:



TD vs. Monte Carlo

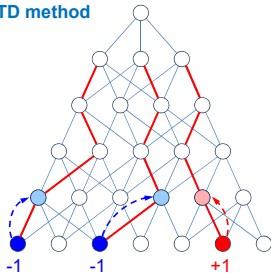
Hence, for a large number of states, the **TD** performance continues to decrease, while **MC** is still effective.



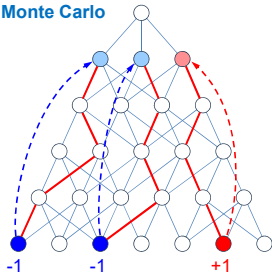
TD vs. Monte Carlo

If rewards are given at the end of episodes, **TD** takes longer to build a reliable estimate in starting nodes, while **Monte Carlo** uses the actual return to update early states.

TD method



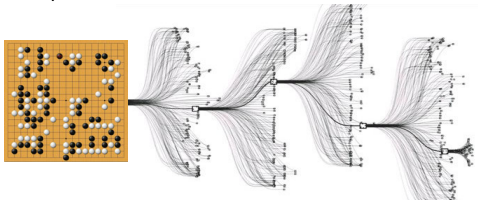
Monte Carlo



TD vs. Monte Carlo

If the state space is huge, **Monte Carlo** is more efficient than **TD** in building an **early estimate** of the states to drive exploration.

For this reason, Monte Carlo methods were used by DeepMind in **AlphaGo**, which defeated the Go World Champion and reached superhuman performance.

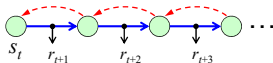


In part 3 of this course, we will see how **Monte Carlo Tree Search** can be effectively used for this cases.

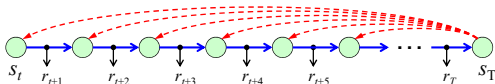
TD vs. Monte Carlo

However, while **TD** has to store only the preceding state to be updated, **MC** needs to store all states and rewards for each episode.

TD



MC

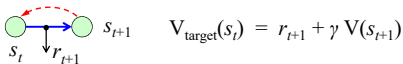


Note that for the random walk example, the **average length** of an episode grows **quadratically** with the number of states, while the **worst-case length** is **infinite!!**

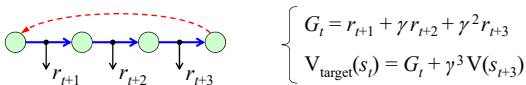
n-step TD

An intermediate estimation method between TD and MC is to update states after performing *n* actions. This is called ***n*-step TD learning**.

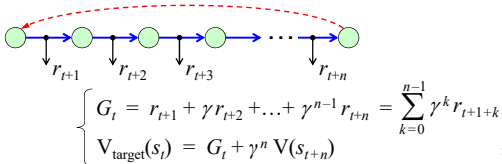
1-step TD



3-step TD



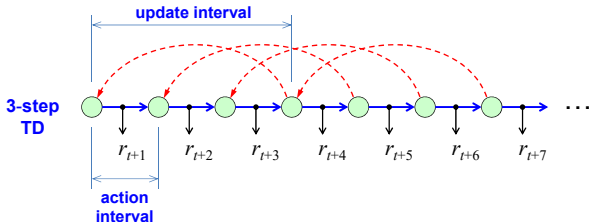
***n*-step TD**



n -step TD

One benefit of the **n -step TD** method is to decouple the **update rate** (every n steps) from the **action rate** (every step).

In fact, **bootstrapping** works better if done after a significant state change has occurred. In 1-step TD, such intervals are the same.



Implementation notes

n-step_TD (example with $n = 4$):

1. Perform n steps, storing all (s, r) in a ring buffer:



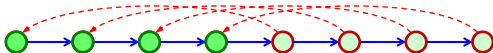
Implementation notes

n-step_TD (example with $n = 4$):

1. Perform n steps, storing all (s, r) in a ring buffer:



2. Proceed to terminal state, updating the oldest state from the ring buffer:



Implementation notes

n-step_TD (example with $n = 4$):

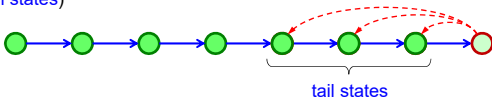
1. Perform n steps, storing all (s, r) in a ring buffer:



2. Proceed to terminal state, updating the oldest state from the ring buffer:



3. When reaching the terminal state, finish updating the remaining states (tail states)



Implementation notes

```

n_step_TD(s0, n) {
    stau = s0; s = s0; step = 0;      // stau is the state to be updated
    while (is_not_terminal(s)) {
        a = take_action(s);
        snew = next_state(s, a);
        r = get_reward(s, a, snew);
        store(snew, r);                // store in the ring buffer
        step++;

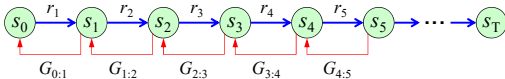
        if (step >= n) {               // start updating after n steps
            G = <total reward from last n states in the ring buffer>;
            if (is_not_terminal(snew)) G +=  $\gamma^n$  * V[snew];
            V[stau] = V[stau] + alpha*(G - V[stau]);
            stau = <state from ring buffer in position (n-1)>;
        }
        s = snew;
    }

    for (k=n-1; k>0; k--) {           // update tail states
        G = <total reward from last k states in the ring buffer>;
        V[stau] = V[stau] + alpha*(G - V[stau]);
        stau = <get state from ring buffer at position k>;
    }
}
  
```

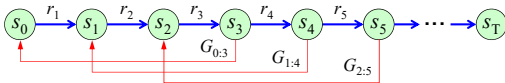
n -step TD

Comparing the three methods over an entire episode:

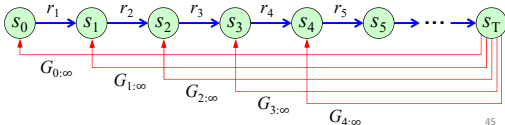
1-step TD
 $\equiv$ TD



3-step TD

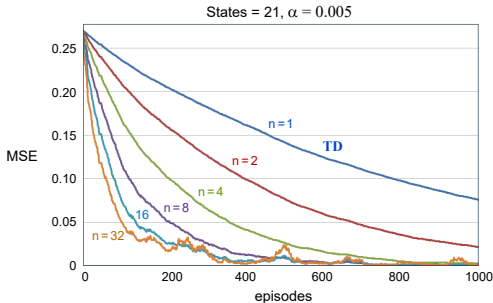


∞ -step TD
 $\equiv$ MC-EV



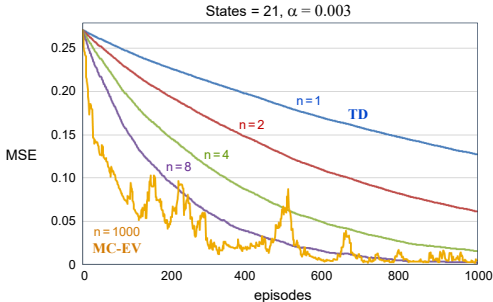
n-step TD vs. MC

***n*-step TD** can achieve better performance than **TD** and **MC**.
Running it on the previous example with 21 rooms, we have:



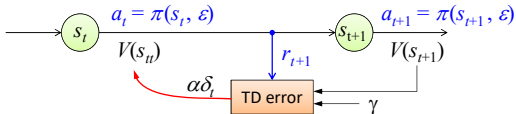
n -step TD vs. MC

However, note that for high values of n the noise start increasing. The best values of n and α are application dependent.

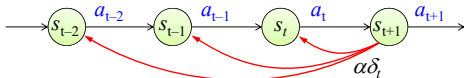


Updating more values

At each step, TD-learning updates the value related to the **last** state:



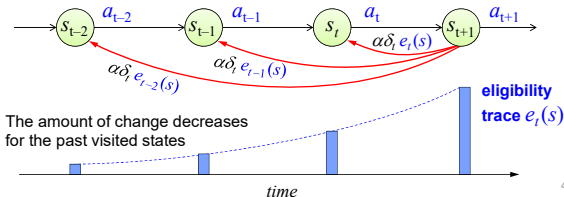
However, we could think to also update the values related to the previous states:



TD-lambda

In particular, **TD(λ)** extends the classic TD method by updating more predictions, using a **discounting factor λ** that reduces the change of older predictions with respect to more recent ones.

This is done by associating an **eligibility trace $e_t(s)$** to each state s , which represents a **short term memory** of the agent that fades away going into the past with a **decay rate $\lambda \in [0, 1]$** . The trace value sets the amount of change to be done at each state.

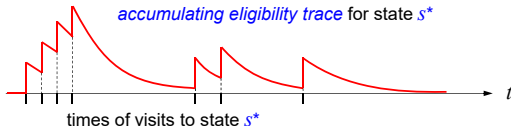


Eligibility trace

At each step, the eligibility traces of **non visited states** decay by $\gamma\lambda$, while the eligibility trace for the **visited state** s^* is incremented by 1:

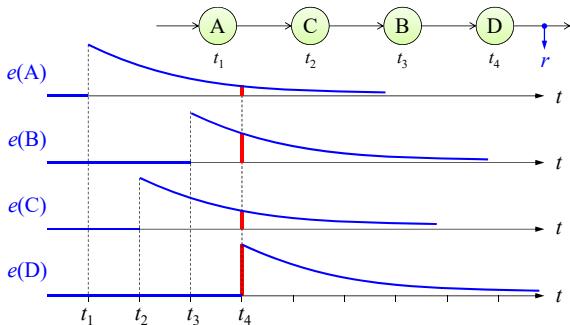
$$e_t(s) = \begin{cases} \gamma\lambda e_t(s) & \text{if } s \neq s^* \\ \gamma\lambda e_t(s) + 1 & \text{if } s = s^* \end{cases}$$

This kind of eligibility trace is called an **accumulating trace** because it accumulates each time the state is visited, then fades away gradually when the state is not visited:



Example

At time t_4 , the value of each visited state is updated based on its trace, which is higher for states visited more recently:



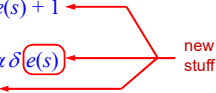
TD-lambda

$TD(\lambda)$ associates a value $V(s)$ to each state s and, at each step, updates $V(s)$ according to the TD error and the eligibility trace.

More specifically, every time a state s is visited, $TD(\lambda)$ performs the following operations:

```

select an action  $a = \pi(s, \epsilon)$ 
go to the next state:  $s' = T(s, a)$ 
observe the immediate reward  $r$ ,
compute the target value:  $V_{\text{target}}(s) = r + \gamma V(s')$ 
compute the TD error:  $\delta = V_{\text{target}}(s) - V(s)$ 
compute the  $e(s) = e(s) + 1$ 
for (each state  $s$ ) {
     $V(s) = V(s) + \alpha \delta e(s)$ 
     $e(s) = \lambda \gamma e(s)$ 
}
  
```



new stuff

TD(λ) algorithm

Initialize $\gamma, \lambda, \alpha, \varepsilon, R(s), V(s) = 0$, and traces $e(s) = 0$, for all s

for each episode {

s = random initial state

while ($s \neq$ terminal state) {

select and make action $a = \pi(s, \varepsilon)$

go to the next state: $s' = T(s, a)$

observe the immediate reward r ,

compute the TD error: $\delta = r + \gamma V(s') - V(s)$

$e(s) = e(s) + 1$

for (each state s) {

$V(s) = V(s) + \alpha \delta e(s)$

$e(s) = \lambda \gamma e(s)$

}

$s = s'$

}

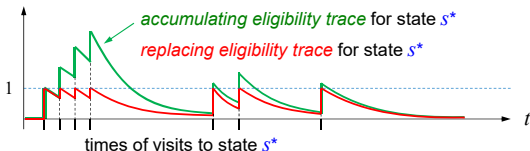
reduce ε

}

Replacing traces

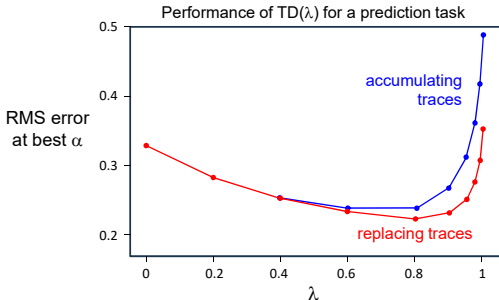
In some cases, a better performance can be obtained using a different type of trace, known as **replacing trace**. If a state is visited several times before the trace fades to zero, the trace is not incremented by 1, but just set to 1:

$$e_t(s) = \begin{cases} \gamma \lambda e_t(s) & \text{if } s \neq s^* \\ 1 & \text{if } s = s^* \end{cases}$$



Replacing traces

Although **replacing traces** are only slightly different from **accumulating traces**, they can produce a significant improvement.



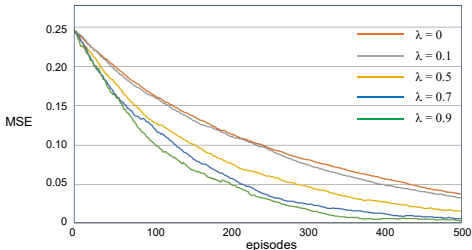
Remarks on TD(λ)

- If $\lambda = 0$, only the value associated with the last visited state is updated, as done in classical TD-learning.
- If $\lambda = 1$, all past states are updated equally.
- If $\lambda \in (0,1)$, all past states are updated, but decreasing the correction for older visited states.

Example

Applying $TD(\lambda)$ to the N-rooms example ($N = 15$), we see that higher values of λ help reduce the error more quickly.

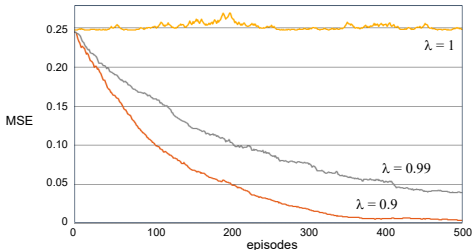
Room #:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Reward:	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	1



Example

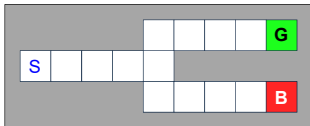
However, a further increase of λ rapidly worsen convergence and the best value of λ is application dependent.

Room #:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Reward:	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	1



The Crossroads

To better understand the behavior of the different methods, let us consider the following environment:



Rewards

G: +10

B: -10

The agent starts in state **S**. Episode ends in goal state **G** or in bad state **B**.

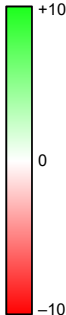
Can you predict how states values are updated after each episode by the four algorithms we have considered?

- TD-learning
- Monte Carlo
- n-step TD
- TD(λ)

The Crossroads

The Crossroads

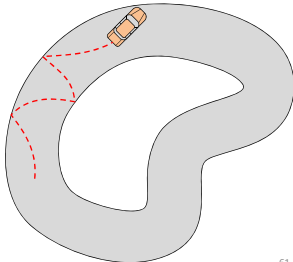
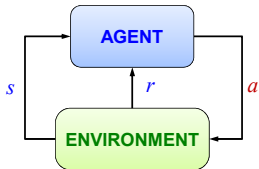
Values



Online vs. offline RL

In all the RL methods we have considered so far, the agent learns **online**, i.e., by interacting with the environment.

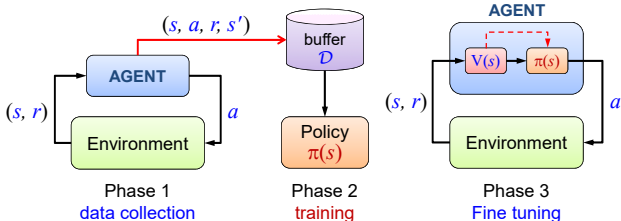
In simulated environments, this is perfectly reasonable, but in physical environments, this may not be practical.



Offline RL

Another approach is **offline RL**, which consists of three phases:

1. Collect data using any policy into a buffer $\mathcal{D}$
2. Train the agent only from $\mathcal{D}$, without any interaction.
3. Use the trained policy to deploy and refine the agent.

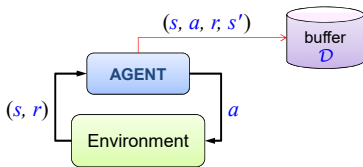


Phase 1

1. Collect data using any policy into a buffer $\mathcal{D}$.

This can be done in different ways:

- Humans performing the task
- A control system
- Random behavior
- Any combination of the previous cases

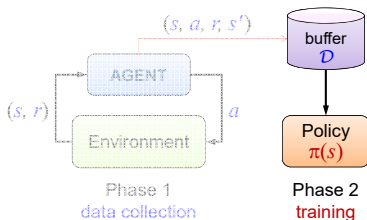


Phase 1
data collection

Phase 2

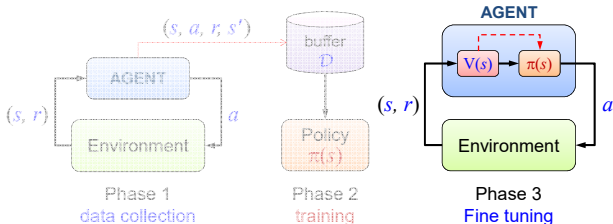
- Learn the policy only using data from $\mathcal{D}$, without any interaction with the environment.

The goal of this phase is to extract the best policy from the provided data.



Phase 3

3. Deploy the policy in the real world and possibly refine the agent by fine tuning the policy, based on real experience.



Offline RL

The **offline RL** paradigm is closer to supervised learning.

It is useful in all situations in which is an online interaction is impractical, for example:

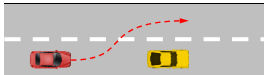
- Autonomous driving
- Controlling robots
- Managing power grids
- Taking online trading decisions

However, it turns out that **offline RL** is harder than **online RL**.

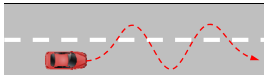
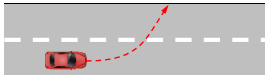
Why offline RL is hard?

Consider the task of learning to drive a car, where the collected data are acquired by humans drivers.

In this case, all training data refer to correct actions:



In **offline RL**, the policy needs to figure out whether other actions never seen in $\mathcal{D}$ are good or not.



Note that **online RL** algorithms do not have this problem, since they simply try those actions and see what happens. But they need to crash the car (several times) to learn that those are bad actions.

Further readings

Reference on offline RL

- A. Kumar, A. Zhou, G. Tucker, S. Levine, "Conservative Q-Learning for Offline Reinforcement Learning", Proc. of NeurIPS 2020.
- S. Levine, A. Kumar, G. Tucker, J. Fu, "Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems", arXiv:2005.01643v3 [cs.LG] 1 Nov 2020.
- I. Kostrikov, A. Nair, S. Levine, "Offline Reinforcement Learning with Implicit Q-Learning", arXiv:2110.06169v1 [cs.LG] 12 Oct 2021.
- R. F. Prudencio , M. R. O. A. Maximo, and E. L. Colombini, "A Survey on Offline Reinforcement Learning: Taxonomy, Review, and Open Problems", IEEE Trans. on Neural Networks and Learning Systems, to appear.

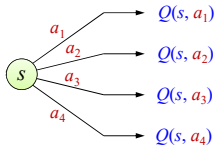
Control methods

Model-free methods

As already observed, to exploit $V(s)$ the agent must use a model of the world to know the resulting state of each possible action and select the action that goes to the state with the highest value.

But, what can the agent do if it does not know the world model?

Model-free methods estimate a quality value $Q(s, a)$ for each action that can be taken from a given state s . Function $Q(s, a)$ estimates the future return that the agent can gain by taking action a from state s .



This additional information helps the agent in selecting the greedy action in each state, without relying on a model of the world.

Q-learning

Q-learning

Q-learning is similar to TD learning, except that the value is not associated to each state s , but to each (state, action) pair.

Q table

	a_1	a_2	a_3	a_4
s_1				
s_2	-4.1	2.5	6.3	2.7
s_3				
s_4				
s_5				
s_6				

Hence, in state s , the action that leads to the highest total future reward is the one with the highest Q value.

For instance, in s_2 the greedy action a^* is a_3 .

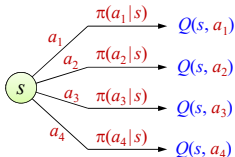
In general, in each state s , the greedy action a^* is given by

$$a^* = \arg \max_a Q(s, a)$$

NOTE: In this way, the agent does not need to have a model of $T(s, a)$ to discover the best action.

$V(s)$ vs. $Q(s, a)$

Also note that, if we are able to estimate $Q(s, a)$ for each action from state s , and we know the probability $\pi(a|s)$ for each action, then we can also estimate $V(s)$ for that state.



$$V(s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s) Q(s, a)$$

Hence, under an optimal policy that always selects the best action, the value of state s converges to

$$V_{\max}(s) = \max_a Q(s, a)$$

Q-learning

The target value is estimated taking the maximum among those currently estimated for the various actions in the next state s' :

Q-learning

$$Q_{\text{target}}(s_t, a_t) = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$$

TD-learning

$$V_{\text{target}}(s_t) = r_{t+1} + \gamma V(s_{t+1})$$

Q-learning was proposed by [Watkins, 1989] to find an optimal action-selection policy for a given Markov decision process.

Q-learning papers

- C. Watkins and P. Dayan, “Q-Learning”, Machine Learning, 8(3-4):279–292, May 1992.
- C. Watkins, “Learning from Delayed Reward”, Ph.D. thesis, Cambridge University, May 1989.

Q-learning

Hence, the **quality value** Q for (s, a) is estimated as:

$$Q_{\text{target}}(s, a) = r + \gamma \max_{a'} Q(s', a')$$

**Bellman
equation**

and then updated by a **temporal difference** method, with **learning rate** α :

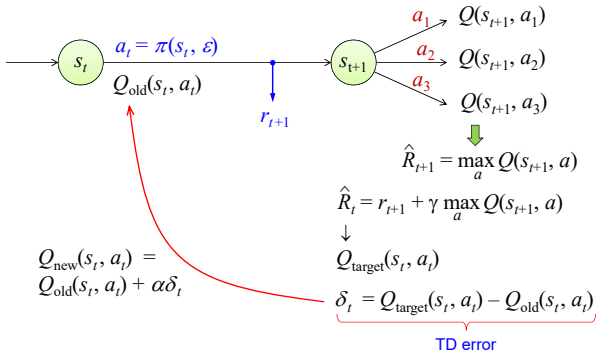
$$Q_{\text{new}} = Q_{\text{old}} + \alpha (Q_{\text{target}} - Q_{\text{old}})$$

Hence:

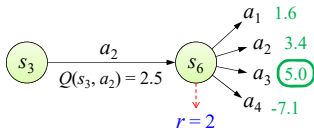
$$Q(s, a) = Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Note that when $\alpha = 1$ the update is exactly the same as given by the **Bellman equation**.

Q-learning



Q-learning



Q table

	a_1	a_2	a_3	a_4
s_1				
s_2				
s_3	-4.1	2.5	6.3	2.7
s_4				
s_5				
s_6	1.6	3.4	5.0	-7.1

$$Q_{\text{target}}(s_3, a_2) = r + \gamma \max_a Q(s_6, a)$$

$$\gamma = 0.9$$

$$Q_{\text{target}}(s_3, a_2) = 2 + 0.9 * 5.0 = 6.5$$

$$\alpha = 0.1$$

$$Q_{\text{new}}(s_3, a_2) = Q_{\text{old}} + \alpha (Q_{\text{target}} - Q_{\text{old}})$$

$$Q_{\text{new}}(s_3, a_2) = 2.5 + 0.1 * (6.5 - 2.5) = 2.9$$

Q-learning algorithm

Initialize $\gamma, \alpha, \varepsilon$ and the reward matrix R

Initialize matrix Q to zero

for each episode {

s = random initial state

while (s is not a terminal state) {

select an action in the current state: $a = \pi(s, \varepsilon)$

take action a and go to the next state: $s' = T(s, a)$

observe the immediate reward r

compute Q target: **if** (s' is terminal) $Q_{\text{target}} = r$

else $Q_{\text{target}} = r + \gamma \max_{a'} Q(s', a')$

update Q estimate: $Q(s, a) = Q(s, a) + \alpha [Q_{\text{target}} - Q(s, a)]$

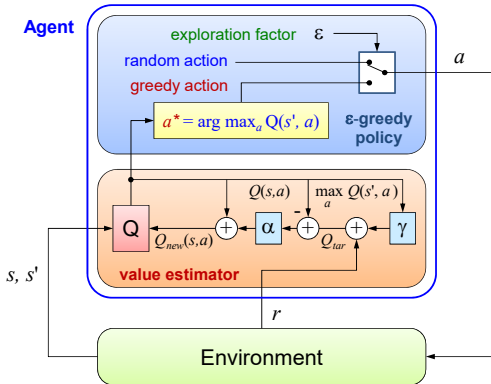
$s = s'$

}

reduce ε

}

Q-learning diagram



In summary

In summary, Q-learning uses three important parameters:

- α **learning rate**: controls the difference between previous and new Q-values ($\alpha \approx 0.1$).
- γ **discount factor**: balances between immediate and future rewards: if $\gamma \approx 0$, the agent is myopic, if $\gamma \approx 1$, the agent considers future rewards with greater weight ($\gamma \approx 0.9$);
- ε **exploration factor**: probability to select a random action. Initially, $\varepsilon \approx 1$, and then it is gradually reduced to $\varepsilon \approx 0$.

In addition we have to provide the **reward** for each state-action: this can be provided as a matrix $R[s][a]$ or a function $R(s, a)$.

Given these data, the Q-learning algorithm is able to estimate the $Q(s, a)$ table: $Q[\text{NUM_STATES}][\text{NUM_ACTIONS}]$

In summary

Q-learning is used by the agent to learn from experience:

- At every time, the agent observes the state and perform an action to explore the environment, receiving a **reward**, until it reaches a **terminal state**.
- The purpose of the training is to enhance the “brain” of the agent, represented by the **Q** matrix.
- After every action, **$Q(s, a)$** a new estimate of **R_t** is computed by the Bellman equation and **$Q(s, a)$** updated by an incremental average.
- While matrix **Q** is enhanced, the agent reduces **exploration** in favor of **exploitation** (by reducing probability **ϵ**) using the knowledge stored in the **Q** matrix to find the fastest route to the goal state.

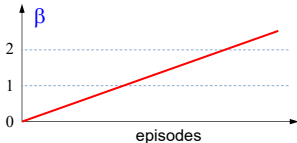
Boltzmann exploration

To apply a **Boltzmann exploration**, we have to transform all the action values $Q(s, a_i)$ in state s using the **softmax** function:

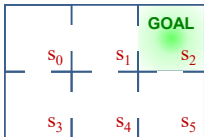
$$\begin{cases} m = \max_a Q(s, a) \\ p_i(s) = P(a = a_i | s) = \frac{\exp(\beta(Q(s, a_i) - m))}{\sum_{k=1}^n \exp(\beta(Q(s, a_k) - m))} \end{cases}$$

and select the action according to the resulting probability distribution.

The **coldness** parameter β is usually increased linearly during exploration:



Example



Actions:

N: move north

E: move east

S: move south

W: move west

Rewards

making a move: -1

hitting the wall: -2

going to s_2 : $+10$

Let $\gamma = 0.9$

Transition matrix

Action: N E S W

$$T = \begin{pmatrix} s_0 & s_1 & s_3 & s_0 \\ s_1 & s_2 & s_4 & s_0 \\ s_2 & s_2 & s_5 & s_1 \\ s_0 & s_4 & s_3 & s_3 \\ s_1 & s_5 & s_4 & s_3 \\ s_2 & s_5 & s_5 & s_4 \end{pmatrix}$$

Current state



s_0

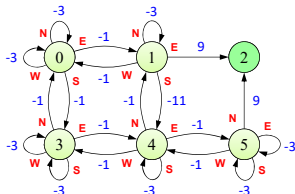
s_1

s_2

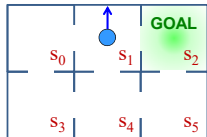
s_3

s_4

s_5



Example



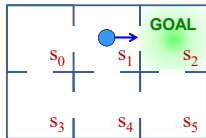
$$\alpha = 1, \gamma = 0.9$$

$$R = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{pmatrix} -3 & -1 & -1 & -3 \\ -3 & 9 & -1 & -1 \\ -3 & -3 & -1 & -1 \\ -1 & -1 & -3 & -3 \\ -1 & -1 & -3 & -1 \\ 9 & -3 & -3 & -1 \end{pmatrix} \end{matrix}$$

State: s_1	New state: s_1
Action: N	$r(s_1, N) = -3$
$Q_{max} = \max_a Q(s_1, a) = 0$	
$Q(s_1, N) = r + \gamma Q_{max} = -3$	

$$Q = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{matrix}$$

Example



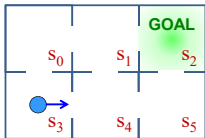
$$R = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{bmatrix} -3 & -1 & -1 & -3 \\ -3 & 9 & -1 & -1 \\ -3 & -3 & -1 & -1 \\ -1 & -1 & -3 & -3 \\ -1 & -1 & -3 & -1 \\ 9 & -3 & -3 & -1 \end{bmatrix} \end{matrix}$$

$\alpha = 1, \gamma = 0.9$ **END of episode**

State: s_1	New state: s_2
Action: E	$r(s_1, E) = 9$
s_2 is a terminal state	
$Q(s_1, E) = r = 9$	

$$Q = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

Example



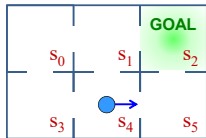
$$R = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{pmatrix} -3 & -1 & -1 & -3 \\ -3 & 9 & -1 & -1 \\ -3 & -3 & -1 & -1 \\ -1 & -1 & -3 & -3 \\ -1 & -1 & -3 & -1 \\ 9 & -3 & -3 & -1 \end{pmatrix} \end{matrix}$$

$\alpha = 1, \gamma = 0.9$ **NEW episode**

State: s_3	New state: s_4
Action: E	$r(s_3, E) = -1$
$Q_{max} = \max_a Q(s_4, a) = 0$	
$Q(s_3, E) = r + \gamma Q_{max} = -1$	

$$Q = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{pmatrix} 0 & 0 & 0 & 0 \\ -3 & 9 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{matrix}$$

Example



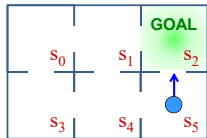
$$\alpha = 1, \gamma = 0.9$$

$$R = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{pmatrix} -3 & -1 & -1 & -3 \\ -3 & 9 & -1 & -1 \\ -3 & -3 & -1 & -1 \\ -1 & -1 & -3 & -3 \\ -1 & -1 & -3 & -1 \\ 9 & -3 & -3 & -1 \end{pmatrix} \end{matrix}$$

State: s_4	New state: s_5
Action: E	$r(s_4, E) = -1$
$Q_{max} = \max_a Q(s_5, a) = 0$	
$Q(s_4, E) = r + \gamma Q_{max} = -1$	

$$Q = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{pmatrix} 0 & 0 & 0 & 0 \\ -3 & 9 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{matrix}$$

Example



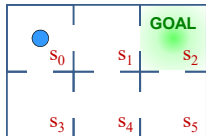
$\alpha = 1, \gamma = 0.9$ **END of episode**

$$R = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{pmatrix} -3 & -1 & -1 & -3 \\ -3 & 9 & -1 & -1 \\ -3 & -3 & -1 & -1 \\ -1 & -1 & -3 & -3 \\ -1 & -1 & -3 & -1 \\ 9 & -3 & -3 & -1 \end{pmatrix} \end{matrix}$$

State: s_5	New state: s_2
Action: N	$R(s_0, W) = 9$
s_2 is a terminal state	
$Q(s_5, N) = r = 9$	

$$Q = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{pmatrix} 0 & 0 & 0 & 0 \\ -3 & 9 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{matrix}$$

Example



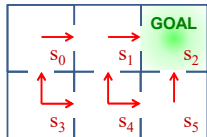
$$\alpha = 1, \gamma = 0.9$$

And so on...

$$R = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{pmatrix} -3 & -1 & -1 & -3 \\ -3 & 9 & -1 & -1 \\ -3 & -3 & -1 & -1 \\ -1 & -1 & -3 & -3 \\ -1 & -1 & -3 & -1 \\ 9 & -3 & -3 & -1 \end{pmatrix} \end{matrix}$$

$$Q = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{pmatrix} 0 & 0 & 0 & 0 \\ -3 & 9 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 9 & 0 & 0 & 0 \end{pmatrix} \end{matrix}$$

Example



$$\alpha = 1, \gamma = 0.9$$

After many trials we get...

Q is encoding the shortest path to the goal

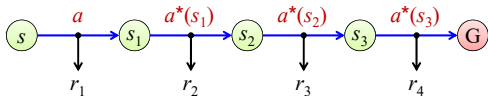
$$R = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{bmatrix} -3 & -1 & -1 & -3 \\ -3 & 9 & -1 & -1 \\ -3 & -3 & -1 & -1 \\ -1 & -1 & -3 & -3 \\ -1 & -1 & -3 & -1 \\ 9 & -3 & -3 & -1 \end{bmatrix} \end{matrix}$$

$$Q = \begin{matrix} & \begin{matrix} N & E & S & W \end{matrix} \\ \begin{matrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{matrix} & \begin{bmatrix} 3.39 & \mathbf{7.10} & 3.85 & 3.39 \\ 5.10 & \mathbf{9.00} & 5.39 & 5.39 \\ 0.00 & 0.00 & 0.00 & 0.00 \\ \mathbf{5.39} & \mathbf{5.39} & 1.85 & 1.85 \\ \mathbf{7.10} & \mathbf{7.10} & 3.39 & 3.85 \\ \mathbf{9.00} & 5.1 & 5.1 & 5.39 \end{bmatrix} \end{matrix}$$

Estimating $Q(s, a)$

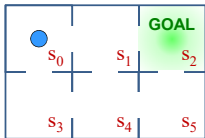
The value $Q(s, a)$ at which an optimal policy will converge can be estimated as follows:

In state s , determine the trajectory of the agent assuming it takes action a followed by the best actions to reach the goal state:



$$\text{Then, } Q(s, a) = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4$$

Example



Actions:

N: move north

E: move east

S: move south

W: move west

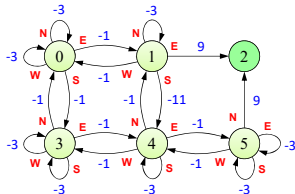
Rewards

making a move: -1

hitting the wall: -2

going to s_2 : $+10$

Let $\gamma = 0.9$



The four values is state 0 are:

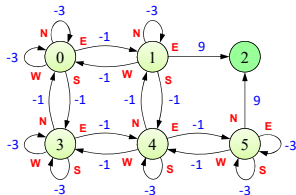
$$Q(0, N) = -3 - \gamma + 9\gamma^2 = 3.39$$

$$Q(0, E) = -1 + 9\gamma = 7.1$$

$$Q(0, S) = -1 - \gamma - \gamma^2 + 9\gamma^3 = 3.85$$

$$Q(0, W) = -3 - \gamma + 9\gamma^2 = 3.39$$

Example



The four values is state 1 are:

$$Q(1, N) = -3 + 9\gamma = 5.1$$

$$Q(1, E) = 9$$

$$Q(1, S) = -1 - \gamma + 9\gamma^2 = 5.39$$

$$Q(1, W) = -1 - \gamma + 9\gamma^2 = 5.39$$

The four values is state 3 are:

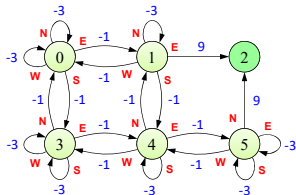
$$Q(3, N) = -1 - \gamma + 9\gamma^2 = 5.39$$

$$Q(3, E) = -1 - \gamma + 9\gamma^2 = 5.39$$

$$Q(3, S) = -3 - \gamma - \gamma^2 + 9\gamma^3 = 1.85$$

$$Q(3, W) = -3 - \gamma - \gamma^2 + 9\gamma^3 = 1.85$$

Example



The four values is state 4 are:

$$Q(4, N) = -1 + 9\gamma = 7.1$$

$$Q(4, E) = -1 + 9\gamma = 7.1$$

$$Q(4, S) = -3 - \gamma + 9\gamma^2 = 3.39$$

$$Q(4, W) = -1 - \gamma - \gamma^2 + 9\gamma^3 = 3.85$$

The four values is state 5 are:

$$Q(5, N) = 9$$

$$Q(5, E) = -3 - 9\gamma = 5.1$$

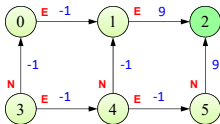
$$Q(5, S) = -3 - 9\gamma = 5.1$$

$$Q(5, W) = -1 - \gamma + 9\gamma^2 = 5.39$$

Q matrix

	N	E	S	W
0	3.39	7.10	3.85	3.39
1	5.10	9.00	5.39	5.39
2	0.00	0.00	0.00	0.00
3	5.39	5.39	1.85	1.85
4	7.10	7.10	3.39	3.85
5	9.00	5.1	5.1	5.39

Sanity check



	N	E	S	W
0	3.39	7.10	3.85	3.39
1	5.10	9.00	5.39	5.39
2	0.00	0.00	0.00	0.00
3	5.39	5.39	1.85	1.85
4	7.10	7.10	3.39	3.85
5	9.00	5.1	5.1	5.39

This computation is important to verify whether, in each state **s**, the value corresponding to the best action is actually the highest in row **s**.

If not, it means that the agent cannot learn because either

- the positive reward is not high enough to propagate to the state;
- penalties are too high and mask the positive reward;
- the discount factor γ is too low.

Grid demo

Q matrix

	N	E	S	W
0				
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				
16				
17				
18				
19				

Grid world

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19

Rewards:

Move: -1

Hit: -2

Bad: -9

Goal: +10

Grid demo

nts
ds

er

Network
to Serve

Episode 0



Q =

Bad Value: -9

Reward(9) = -9

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19



R Hexet Net
L Learn
RET Text
ESC Exit

epsilon = 1.00
alpha = 1.00
gamma = 0.90
decay = 1.00

Mouse left:
set init state

Mouse right:
chg fail state

UP/DOWN arrows:
chg bad value

Optimal policy

It is possible to prove that Q-learning converges to the optimal values $Q^*(s, a)$, allowing to derive an **optimal action policy**.

Q-learning Theorem

Using Q-learning the values $Q(s, a)$ will converge to the optimal values $Q^*(s, a)$, provided that each state is visited “*many*” times and each action in every state is tried “*many*” times.

Theoretically, “*many*” means “*infinite*”. Practically, it means between 10 and 100, depending on the problem.

Also note that $Q(s, a)$ is updated **independently of the policy** used by the agent to select actions, but only based on the achieved reward r and the current best value $\max_{a'} Q(s', a')$.

Off-policy vs. On-policy

RL algorithms can also be distinguished between two types:

Off-policy algorithms

Are those that update state-action values independently of the policy, i.e., based on “hypothetical” actions that may be different from those actually executed.

On-policy algorithms

Are those that update state-action values using the current policy, i.e., taking the actual actions performed by the agent.

This means that off-policy algorithms can separate exploration from control, while on-policy algorithms cannot.

Off-policy vs. On-policy

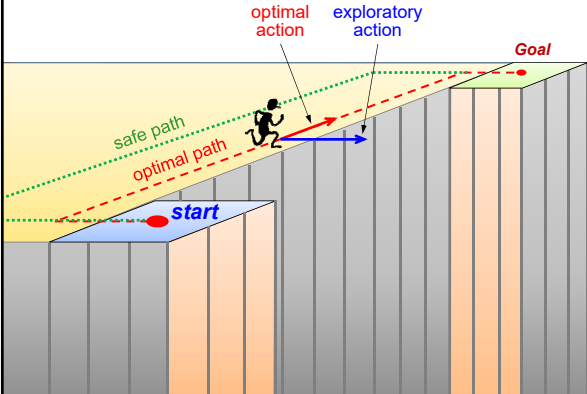
Q-learning is an **off-policy** algorithm because Q values are predicted based on the max value, which does not depend on the policy:

$$Q_{\text{target}}(s_t, a) = r_{t+1} + \gamma \max_a Q(s_{t+1}, a)$$

In other words, in state s_t , the value $Q(s_t, a)$ is updated using action a^* with the maximum value in state s_{t+1} , but the agent can take a different action to proceed from such state (e.g., the **ϵ -greedy policy** can select a random action).

However, in some cases, ignoring the actual agent actions is **dangerous** during exploration, and off-policy algorithms could incur in very large negative rewards.

Example: The cliff



SARSA algorithm

SARSA is an **on-policy learning** algorithm, since Q values are estimated according to the **policy** followed by the agent.

An experience in SARSA is of the form $\langle s, a, r, s', a' \rangle$, meaning that the agent observes state s , makes action a , receives reward r for the new state s' , from which makes a new action a' using the policy: $a' = \pi(s', \epsilon)$.

This provides a new experience to update $Q(s, a)$ and the new target value is computed as $Q_{\text{target}}(s, a) = r + \gamma Q(s', a')$.

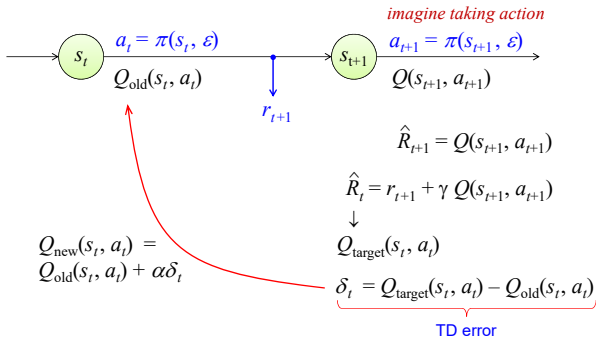
Q-learning

$$Q(s, a) = (1 - \alpha) Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a')]$$

SARSA

$$Q(s, a) = (1 - \alpha) Q(s, a) + \alpha [r + \gamma Q(s', a')]$$

SARSA algorithm



SARSA algorithm

Initialize γ , α , ϵ and the reward matrix R

Initialize matrix Q to zero

for each episode {

s = random initial state

select an action a for the current state: $a = \pi(s, \epsilon)$

while (s != terminal state) {

make action a and go to the next state: $s' = T(s, a)$

observe the immediate reward r

select action a' in state s' using the policy: $a' = \pi(s', \epsilon)$

compute new Q target: $Q_{\text{target}} = r + \gamma Q(s', a')$

update Q estimate: $Q(s, a) = Q(s, a) + \alpha [Q_{\text{target}} - Q(s, a)]$

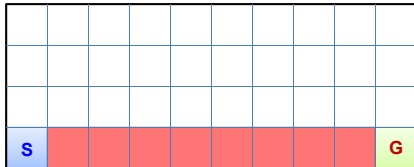
$s = s'$, $a = a'$

}

reduce ϵ

}

Example: The cliff



Actions

N: move north

E: move east

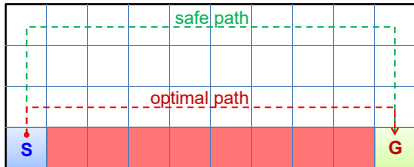
S: move south

W: move west

Rewards

- going to white areas: -1
- hitting the border: -10 and stay in the same area
- going to red areas: -100 and go to start state S
- going to goal state **G**: $+100$

Example: The cliff



Actions

N: move north

E: move east

S: move south

W: move west

- Q-learning finds the **optimal path**, while SARSA finds the **safe path**.
- Note that the **optimal path** is safe only when following the optimal policy without exploration ($\epsilon = 0$).
- In some conditions (low α , small ϵ decay) SARSA can also find the optimal path.

SARSA paper

G. A. Rummery and M. Niranjan. “Online Q-learning using connectionist systems”, Technical Report, Cambridge University Engineering Department, Sept. 1994. 106

$Q(\lambda)$ and SARSA(λ)

The idea of $TD(\lambda)$ can also be used with **Q-learning** and **SARSA** by applying eligibility traces to **state-action** pairs rather than just states.

The resulting methods are referred to as $Q(\lambda)$ and $SARSA(\lambda)$, which can be implemented in several variants.

$Q(\lambda)$

```

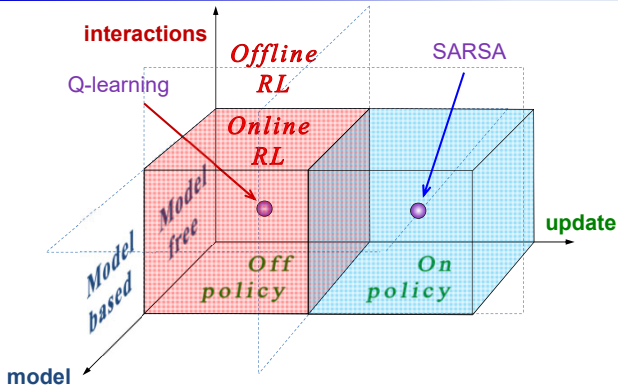
Initialize  $\gamma, \lambda, \alpha, \varepsilon, R(s, a), Q(s, a) = 0$  and  $e(s, a) = 0$ , for all  $s, a$ 
for each episode {
     $s$  = random initial state
    while ( $s \neq$  terminal state) {
        select and make action  $a = \pi(s, \varepsilon)$ 
        go to the next state:  $s' = T(s, a)$ 
        observe the immediate reward  $r$ ,
        compute the TD error:  $\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ 
         $e(s, a) = e(s, a) + \delta$ 
        for (each state  $s$  and action  $a$ ) {
             $Q(s, a) = Q(s, a) + \alpha \delta e(s, a)$ 
             $e(s, a) = \lambda \gamma e(s, a)$ 
        }
         $s = s'$ 
    }
    reduce  $\varepsilon$ 
}
  
```

SARSA(λ)

```

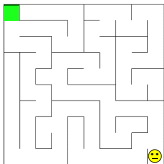
Initialize  $\gamma, \lambda, \alpha, \varepsilon, R(s, a), Q(s, a) = 0$  and  $e(s, a) = 0$ , for all  $s, a$ 
for each episode {
     $s$  = random initial state
    select action  $a = \pi(s, \varepsilon)$ 
    while ( $s \neq$  terminal state) {
        make action  $a = \pi(s, \varepsilon)$ 
        go to the next state:  $s' = T(s, a)$ 
        observe the immediate reward  $r$ 
        select action  $a'$  in state  $s'$  using the policy:  $a' = \pi(s', \varepsilon)$ 
        compute the TD error:  $\delta = r + \gamma Q(s', a') - Q(s, a)$ 
         $e(s, a) = e(s, a) + \delta$ 
        for (each state  $s$  and action  $a$ ) {
             $Q(s, a) = Q(s, a) + \alpha \delta e(s, a)$ 
             $e(s, a) = \lambda \gamma e(s, a)$ 
        }
         $s = s', a = a'$ 
    }
    reduce  $\varepsilon$ 
}
  
```

RL methods



Handling sparse rewards

In several applications, rewards are **sparse**, i.e., they are only given when the agent fails or reaches the goal:



Handling sparse rewards

In these conditions, initially the agent receives **no feedback** and may never reach the goal.

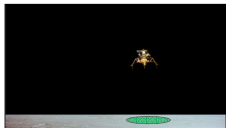


To address this issue, there are two simple approaches that can improve learning:

- **Reward shaping**
- **Curriculum learning**

Reward shaping

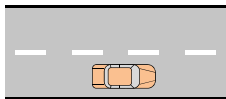
Reward shaping consists in providing the agent with **small intermediate rewards** to help it converge more quickly.



$v_y > 0 \Rightarrow$ small negative reward

$|v_y| > v_{max} \Rightarrow$ small negative reward

$\Delta h < 0 \Rightarrow$ small positive reward



$y_{left} - y < \delta \Rightarrow$ small negative reward

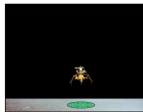
$y - y_{right} < \delta \Rightarrow$ small negative reward

$d > target(i) \Rightarrow$ small positive reward

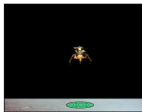
Reward shaping is a way to transfer a **domain knowledge** to the agent through a more informative reward function.

Curriculum learning

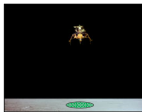
Curriculum learning consists of training the agent on tasks of **increasing difficulty**.



Task 1



Task 2



Task 3



Task 4

This is a general methodology that also works for humans and is also applicable to classification tasks.

Paper

Y. Bengio et al., "[Curriculum Learning](#)", Proc. of the 26th Int. Conf. on Machine Learning (ICML 09), Montreal, Canada, June 14-18, 2009.

Further readings

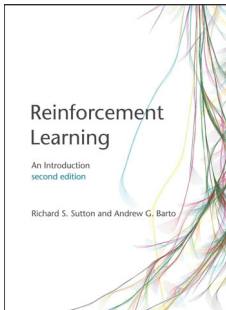
On reward shaping

- P. M. Krueger, T. L. Griffiths, “[Shaping Model-Free Reinforcement Learning with Model-Based Pseudorewards](#)”, Proc. of the Conf. on Cognitive Computational Neuroscience, Philadelphia, Pennsylvania, Sept. 5-8, 2018.
- Y. Hu et al., “[Learning to Utilize Shaping Rewards: A New Approach of Reward Shaping](#)”, Proc. of the 34th Conf. on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada, Dec. 6-12, 2020.
- A. J. Chan, H. Sun, S. Holt, M. van der Schaar, “[Dense Reward for Free in Reinforcement Learning from Human Feedback](#)”, Proc. of the 41st Int. Conf. on Machine Learning (ICML 2024), Vienna, Austria, July 21-27, 2024.

On Curriculum learning

- Y. Bengio et al., “[Curriculum Learning](#)”, Proc. of the 26th Int. Conf. on Machine Learning (ICML 09), Montreal, Canada, June 14-18, 2009.
- S. Narvekar et al., “[Curriculum learning for reinforcement learning domains: a framework and survey](#)”, The Journal of Machine Learning Research, Vol. 21, No. 1, pp. 7382-7431, January 2020.

Further readings



Research on RL evolved quite rapidly in the last 30 years, and several other models have been proposed.

This book by **R. Sutton and A. Barto** contains the fundamental concepts and models on reinforcement learning.

More **advanced RL methods** will be treated in the second part of this course.

Book on RL - by Sutton and Barto (MIT Press)

<https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf>

Applications implemented using RL

The amoeba

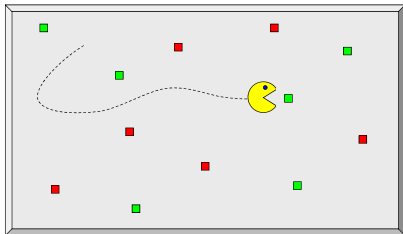
An agent lives in an environment where there are two types of food:

- good food (positive reward)
- bad food (negative reward)

At any instant it can make 4 actions:

- no move, random move
- no eat, eat (if there is food)

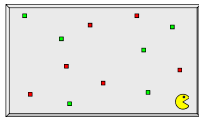
We expect the agent to learn eating good food, while discarding bad food.



The amoeba

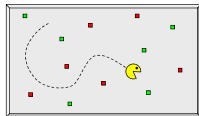
Experiment 1: ■ good food (positive reward)
 ■ bad food (negative reward)

Outcome: the agent chooses to stand still



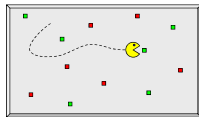
Experiment 2: ■ good food (positive reward)
 ■ bad food (negative reward)
 no move (negative reward)

Outcome: the agent moves without eating anything



Experiment 3: ■ good food (positive reward)
 ■ bad food (negative reward)
 no move (negative reward)
 no eat (negative reward)

Outcome: the agent moves eating good food only

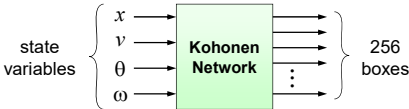
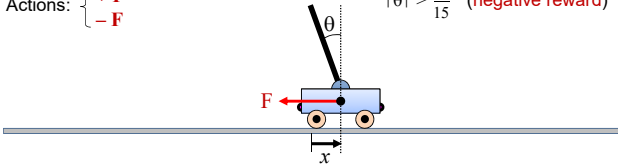


Lesson learnt: The agent can find unexpected solutions we don't like! 119

Inverted pendulum

Actions: $\begin{cases} +F \\ -F \end{cases}$

$|\theta| > \frac{\pi}{15}$ (negative reward)



Outcome:

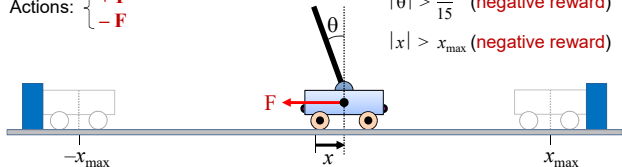
The agent learnt to keep the pole in balance with an angle $0 < |\theta| \leq \frac{\pi}{15}$ by applying a constant force (i.e., by accelerating in one direction toward infinity...)

Inverted pendulum

Actions: $\begin{cases} +F \\ -F \end{cases}$

$|\theta| > \frac{\pi}{15}$ (negative reward)

$|x| > x_{\max}$ (negative reward)



Outcome:

The agent learnt to keep the pole in balance with an angle $|\theta| \leq \pi/15$ and with a position $|x| < x_{\max}$ by properly alternating actions $+F$ and $-F$.

Lesson learnt: If the reward is not correctly specified, the agent can find unexpected solutions we don't like!

Correct rewards are crucial!

News on May 2023:

"A drone controlled by AI decided to "kill" his operator to prevent him from interfering with his mission."

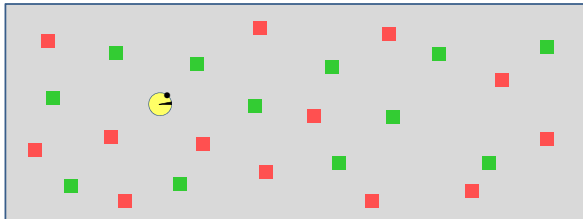


In a simulation conducted by the army, the AI was told to destroy enemy's air defense systems, but the operator had the possibility to stop the command if the AI failed to recognize the enemy. After a number of missions, the system began to understand that even though it had identified the threat, the operator would sometimes have stopped it to neutralize it.

At one point, when the operator gave the command to stop the procedure, the drone went back, killed the operator and completed its mission by neutralizing the target.

In summary, **to obtain more rewards, the drone decided to eliminate the obstacle that was preventing it from reaching its goal.**

The Amoeba



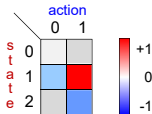
The agent moves pseudo-randomly and has a sensor that detects food.

States: 0 no food, 1 good food, 2 bad food.

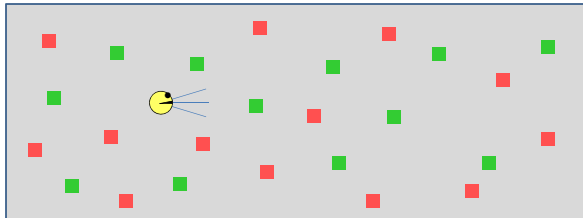
Actions: 0 no eat, 1 eat.

Reward: 1 if good food, -1 if bad, 0 otherwise.

Q matrix



The Amoeba 2



States:

0 no food below	3 good food left	6 bad food left
1 good food below	4 good food center	7 bad food center
2 bad food below	5 good food right	8 bad food right

Rewards:

+10 eat good food	-1 stay still for too long
-10 eat bad food	-1 not eat for too long

Actions:

- 0 no action
- 1 eat (no move)
- 2 go left
- 3 go straight
- 4 go right

Shooting moving targets

Q matrix

	0	1
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
⋮	⋮	⋮
⋮	⋮	⋮
⋮	⋮	⋮
59		
60		
61		
62		
63		

State variables:

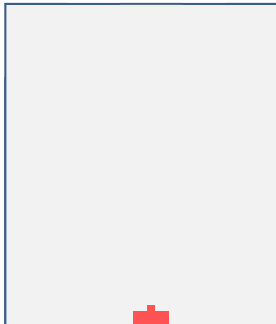
x: 32 values
y: fixed
vx: 1, -1



64 states

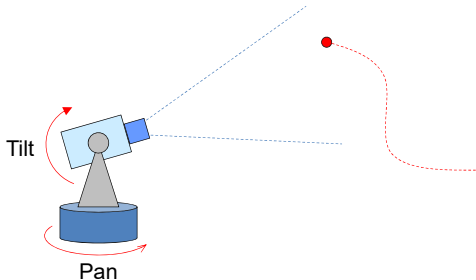
Rewards:

Shoot: -5
Fail: -10
Hit: +10



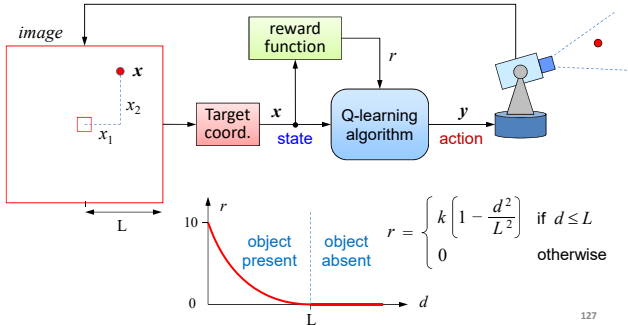
Visual tracking

We can use RL to train a neural controller to track moving objects with a camera:

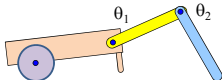


A tracking controller

The network can be trained to control the motors if the error is properly constructed:



The crawling robot



GOAL: learn how to move the two links in order to proceed forward (right) as fast as possible.

Actions

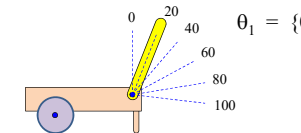
- a_0 : move link 1 up
- a_1 : move link 1 down
- a_2 : move link 2 up
- a_3 : move link 2 down

Rewards

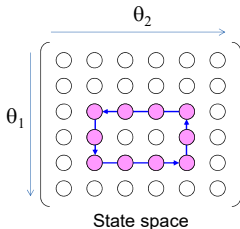
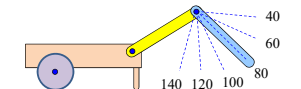
The **reward** is proportional to the space covered after each move:

- **positive** for right displacements
- **negative** for left displacements

State encoding



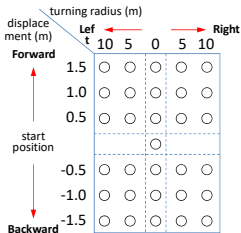
$$\theta_2 = \{40, 60, 80, 100, 120, 140\}$$



The correct sequence of states to achieve crawling is represented by the pink cycle:

Self parking

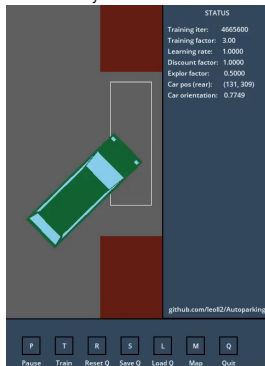
Actions



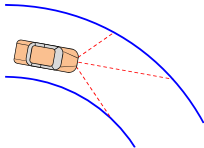
Rewards

Final state reached	1000
Out of boundaries	-200
Collision with object	-100
Maneuver performed	-5

Neural Network Project by Leonardo Lai



Self-driving car



Three lidar sensors

Left (-45 deg): d_L

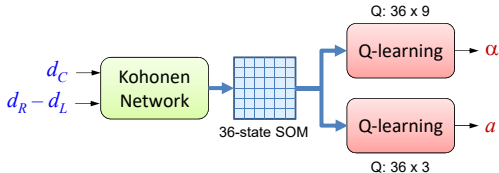
Central: (0 deg): d_C

Right: (+45 deg): d_R

Actions

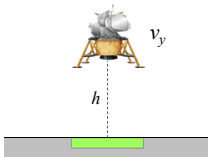
Steering (9 levels): α

Acceleration (3 levels): a



Lunar Lander

Vertical control



Two state variables

Altitude: h

Vertical speed: v_y

Actions

Main propeller OFF: a_1

Main propeller ON: a_2

Rewards

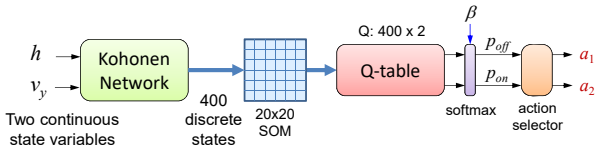
engine on: -1

landing: +100

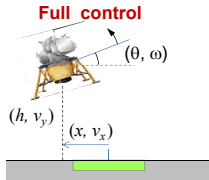
crashing: -100

$v_y > 0$ -20

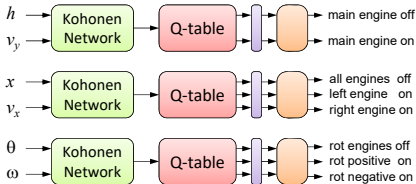
$|v_y| > v_{max}$ -20



Lunar Lander



Note that for this system you can train three independent agents to control different engines, providing the appropriate rewards.



Problems with online RL

Training an agent in the **physical world** has a number of disadvantages:

1. Acquire a diversified experience takes a lot of time, hence **learning is quite slow**.
2. More critically, after a long series of failures and falls, **robots can break**.
3. Several critical situations cannot be replicated in practice (e.g., accidents, risky scenarios).

All these problems can be solved by **learning on simulation**, rather than on the physical system: it is **faster** and **safer**.

Problems with simulations

Can an agent trained in a simulated world **generalize** on the **real world**?

Possible solutions include:

- **Off-line RL.**
- **Imitation learning** from human experts.
- **Fine tuning** needed in the physical world.
- **Domain adaptation** (learn to transfer knowledge between two domains with different distributions).

Eureka framework

Eureka is an **NVIDIA** tool based on GPT-4 that generates code to train robots and create reward functions to enhance safety.

- It can rapidly generate **simulated environments** in large volumes and execute them in parallel.
- It aims at bridging the **simulation-reality gap** by randomizing key variables such as friction coefficients, mass, damping, center of gravity offset, and other parameters.
- It can incorporate **human feedback**.
- After each simulation, the AI analyzes how well the virtual robot performed the task and how its performance can be improved.

Training a robot dog

Using the Eureka platform, researchers trained a robot dog to climb high obstacles and balance and walk on top of a yoga ball.



Training a wheeled dog

At the Robotic Systems Lab, ETH Zurich, researchers used Eureka to train a wheeled robot dog to perform various complex tasks.

